Task 1: Build a User Registration System

Description: Develop a user registration system that allows users to sign up, log in, and view their profile.

Task Documentation:

Database Setup:

- Set up a database to store user information using a technology like MySQL or MongoDB.

Back-End User Registration API:

- Create back-end API endpoints for user registration, using technologies like Node.js and Express.

Data Validation:

- Validate user input on the server-side to ensure data integrity.

Password Hashing:

- Implement password hashing to securely store user passwords.

Front-End Registration Form:

- Design and create a front-end registration form using HTML and CSS.

Front-End Interaction:

- Use JavaScript to send user registration data to the server via API calls.

Back-End User Login API:

- Create API endpoints for user login, session management, and authentication.

User Profile Page:

- Develop a user profile page that displays user information fetched from the server.

Logout Functionality:

- Implement a logout mechanism that destroys the user session.

Testing and Deployment:

- Test the user registration system's functionality and deploy it to a server.

Task 2: Create a Simple To-Do List Application

Description: Build a full-stack to-do list application that allows users to add, complete, and remove tasks.

Task Documentation:

Front-End Design:

- Create a front-end layout with HTML and CSS for the to-do list.

Front-End Interaction:

- Use JavaScript to enable adding, completing, and removing tasks.

Front-End API Requests:

- Use fetch or AJAX to send requests to the back-end API.

Back-End API Endpoints:

- Set up back-end API endpoints for adding, completing, and deleting tasks.

Database Integration:

- Integrate a database to store tasks, using technologies like MongoDB or SQLite.

Task Data Model:

- Define the data model for tasks and implement CRUD operations.

Front-End Task Display:

- Display tasks retrieved from the server on the front-end.

Task Completion Logic:

- Implement functionality to mark tasks as completed or uncompleted.

Testing and Debugging:

- Test the application's functionality thoroughly and debug any issues.

Deployment:

- Deploy the to-do list application to a web server or hosting platform.

Task 3: Build a Blogging Platform

Description: Develop a simple blogging platform with user authentication, post creation, and commenting features.

Task Documentation:

Database Setup:

- Set up a database to store user information, blog posts, and comments.

User Authentication:

- Implement user registration and login functionality using a back-end framework.

Front-End User Authentication:

- Create front-end forms and pages for user registration and login.

Blog Post Creation:

- Design and create a front-end form for users to write and submit blog posts.

Back-End API Endpoints:

- Create API endpoints for creating, fetching, and displaying blog posts.

Commenting System:

- Develop a system for users to comment on blog posts and display comments.

User Profile Pages:

- Build user profile pages that showcase a user's authored posts and comments.

Front-End Post Display:

- Design the front-end layout to display blog posts and comments.

Testing and Debugging:

- Thoroughly test the platform's functionality and address any issues.

Deployment:

- Deploy the blogging platform to a web server or hosting platform.

Task 4: Create a Basic E-Commerce Website

Description: Develop a basic e-commerce website with product listings, cart functionality, and checkout process.

Task Documentation:

Product Listings:

- Create a front-end layout to display product listings with images and descriptions.

Front-End Cart:

- Design a shopping cart that allows users to add and remove products.

Front-End Interaction:

- Use JavaScript to enable adding products to the cart and updating quantities.

Back-End API Endpoints:

- Create API endpoints to retrieve product information and manage cart items.

Database Integration:

- Set up a database to store product details and cart information.

Checkout Process:

- Design a checkout process that allows users to enter shipping and payment information.

Payment Integration (Optional):

- Integrate a payment gateway to process payments securely.

Order Confirmation:

- Provide users with an order confirmation page after successful checkout.

Front-End Order History:

- Create a page for users to view their order history and status.

Testing and Deployment:

- Test the e-commerce website's functionality and deploy it to a server.

Task 5: Build a Real-Time Chat Application

Description: Develop a real-time chat application using technologies like WebSocket and Node.js.

Task Documentation:

Backend Setup:

- Set up a back-end server using Node.js and a WebSocket library.

Front-End Design:

- Design a chat interface using HTML and CSS.

Front-End Interaction:

- Use JavaScript to handle sending and receiving messages.

WebSocket Implementation:

- Implement WebSocket communication for real-time messaging.

User Authentication (Optional):

- Integrate user authentication to identify users in the chat.

Message Storage (Optional):

- Store messages temporarily or persistently, depending on requirements.

Multiple Chat Rooms (Optional):

- Implement the ability to join and switch between different chat rooms.

Testing and Debugging:

- Thoroughly test the chat application's real-time functionality.

Deployment:

- Deploy the chat application to a web server or hosting platform.

Documentation and Scalability Considerations:

- Document the chat application's architecture and consider scalability.

Task 6: Build a Basic Online Quiz Platform

Description: Develop a basic online quiz platform that allows users to take quizzes and view their scores.

Task Documentation:

Database Setup:

- Set up a database to store quiz questions, options, and user scores.

Quiz Question Entry:

- Create a form to allow administrators to enter quiz questions and answer options.

Back-End API Endpoints:

- Develop API endpoints for fetching quiz questions, submitting answers, and calculating scores.

Front-End Quiz Interface:

- Design a user-friendly interface for taking quizzes.

User Authentication:

- Implement user registration and login to track quiz scores.

User Score Display:

- Display the user's quiz scores on their profile page.

Front-End Interaction:

- Use JavaScript to handle quiz interactions, answer submission, and score calculation.

Quiz Results Feedback:

- Provide feedback to users on their quiz results, including correct answers.

Testing and Debugging:

- Test the quiz platform's functionality thoroughly and address any issues.

Deployment:

- Deploy the online quiz platform to a web server or hosting platform.